

AMRITA VISHWA VIDYAPEETHAM
A M R I T A P U R I

B. Tech – Computer Science Engineering (Artificial Intelligence)
22AIE401 – Reinforcement Learning

CardChess-RL

Learning Card Selection in a Rule-Modifying Chess Variant via Self-Play

Team No: 9

Sections 9 – 14

RL Formulation · Solution Diagram · Simulation Plan · Evaluation · Novelty · References

Dhruv Nair
AM.SC.U4AIE23501

Ihsal Riyas
AM.SC.U4AIE23037

Dano Aby Alex
AM.SC.U4AIE23029

9. Initial RL Formulation: State, Action, Reward, Policy, Objective

9.1 MDP Formulation

We model the card-selection problem in Card-Chess RL as a Semi-Markov Decision Process (Semi-MDP) as the agent does not act at every game step, i.e., the agent only acts on rounds where the coin toss designates it as the picker. The time between successive decisions is stochastic and variable.

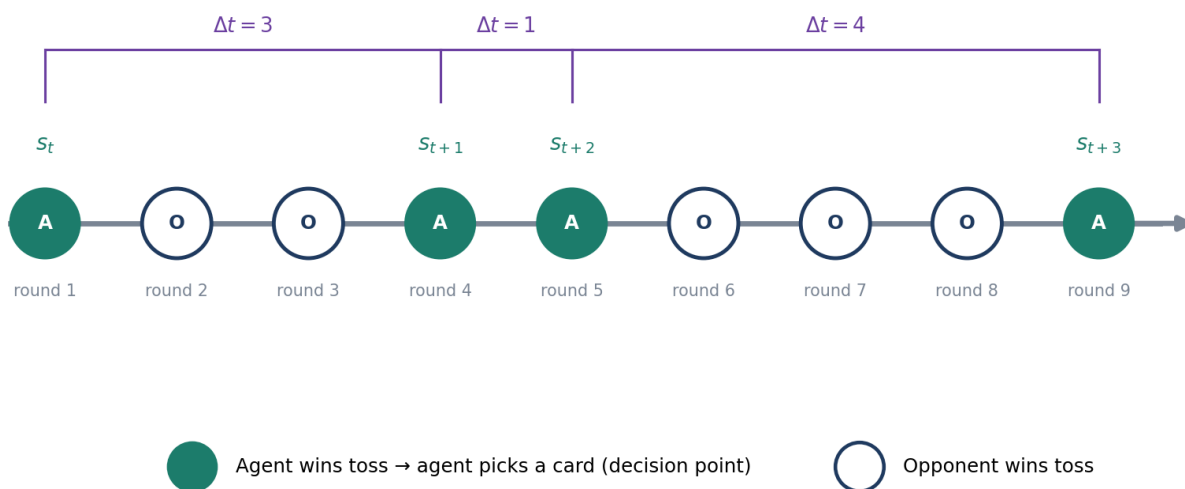


Figure 1. Semi-MDP timeline: the agent (A) acts only on rounds it wins the coin toss; Δt varies between decisions.

9.2 State Space (S)

The state s_t observed by the agent at decision point t is a composite of three components:

Board Encoding (AlphaZero-style spatial planes):

- 12 binary 8×8 planes — 1 per piece type and colour combination — representing the current position on the board.
- 1 binary 8×8 plane representing the turn (whose move it is).
- 2 binary 8×8 planes for castling (each colour separately: kingside or queenside).
- 1 binary 8×8 plane for en-passant square (if applicable).

Total number of board planes: 16 planes of 8×8 each (i.e., 1,024 features).

Candidate Card Encoding:

Each of the 3 candidate cards available to the agent is represented as a fixed-size feature vector:

- Piece type target (one-hot among 6 piece types): the type of piece that is affected by the card.
- Effect type (one-hot among various effect categories: increases mobility, decreases mobility, grants immunity from attacks, etc.).
- Magnitude or scope of the effect (scalar).

Dimensionality of each card vector is d_c ; three vectors are combined into $3 \times d_c$ matrix. **Deck State Encoding:**

- A 15-dimensional binary vector indicating which cards remain in the deck (1 = still available, 0 = already used).
- This gives the agent partial information about what rule changes may appear in the future.

Formal state:

$$s_t = (B_t, C_t^{(1)}, C_t^{(2)}, C_t^{(3)}, d_t)$$

where B_t is the board tensor, $C_t^{(i)}$ is the feature vector for the i -th candidate card, and $d_t \in \{0, 1\}^{15}$ is the deck state.

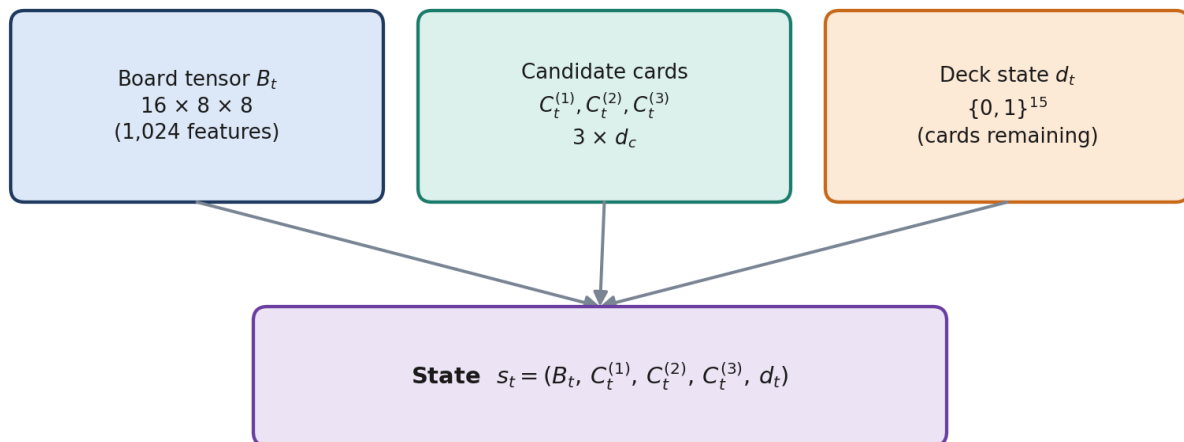


Figure 2. Composition of the state s_t observed at each decision point.

9.3 Action Space (A)

The action space is discrete and fixed at 3:

$$A = \{1, 2, 3\}$$

corresponding to choosing the 1st, 2nd, or 3rd provided card. This value is very low because the task assigned to the RL algorithm is to make the selection of cards while leaving the task of choosing chess moves to the classical search algorithm of Module 3.

9.4 Transition Dynamics (T)

Transition $s_t \rightarrow s_{t+1}$ is determined by a series of events that take place between two consecutive agent's choices:

1. Agent chooses card at; the rule change associated with this card becomes active.
2. Both players make one move according to the new rule set (move choices made by the minimax engine, Module 3).
3. One or several rounds can occur when the opponent wins the coin flip (agent doesn't play during this time).
4. Once the agent wins the coin flip again, the board position, the new set of candidate cards, and the new deck will constitute state s_{t+1} .

Since there is a coin flip and the opponent can choose his/her own cards, this process is stochastic and the number of game rounds between s_t and s_{t+1} is uncertain – which makes this problem a Semi-MDP.

9.5 Reward Function (R)

The reward signal combines a sparse terminal reward with a dense potential-based shaping term:

Terminal reward:

$$R_{\text{terminal}} = +1 \text{ (win)} / 0 \text{ (draw)} / -1 \text{ (loss)}$$

Potential-based shaping reward (Ng, Harada & Russell, 1999):

$$r_{\text{shape}}(s_t, s_{t+1}) = \gamma^{\Delta t} \cdot \Phi(s_{t+1}) - \Phi(s_t)$$

Here, $\Phi(s)$ denotes the evaluation function of the board learned in Module 3 (material balance and positional heuristics), while Δt refers to the number of game moves that have passed between t and $t+1$.

The introduction of $\gamma\Delta t$ in place of a constant γ is the modification introduced to cater to the irregular length of the interval introduced by the coin flip scheme.

Combined return at decision d_j (out of k total decisions in a game):

$$G(d_j) = \gamma^{k-j} \cdot R_{\text{terminal}} + \beta \cdot \sum_{j'=j}^k \gamma^{j'-j} \cdot r_{\text{shape}}(S_{j'}, S_{j'+1}), \quad \beta \approx 0.1-0.3$$

9.6 Policy (π)

The policy $\pi_\theta(a|s)$ is a parameterised stochastic policy implemented as a neural network:

$$\pi_\theta(a|s) = \text{softmax}(f_\theta(s))_a, \quad a \in \{1, 2, 3\}$$

$f_\theta(s)$ generates 3 logits. The network architecture (as described in Solution Diagram in Section 10) consists of convolutional layers on the board planes combined with the card and deck feature vectors and fed into the MLP head.

$V_\phi(s)$ is a value head, which shares the same trunk with $f_\theta(s)$ and generates a scalar representing the expected return from state s . It is used to compute advantage functions.

9.7 Objective

The training objective is to maximise expected return under the policy:

$$J(\theta) = E_{\pi_\theta} [\sum_{j=1}^k G(d_j)]$$

In practice, this is optimised using PPO's clipped surrogate objective (Schulman et al., 2017):

$$L^{\text{CLIP}}(\theta) = E_t [\min(r_t(\theta) \cdot \hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \cdot \hat{A}_t)]$$

where $r_t(\theta) = \pi_\theta(a_t|s_t) / \pi_{\theta_{\text{old}}}(a_t|s_t)$ is the probability ratio, $\hat{A}_t = G(d_t) - V_\phi(s_t)$ is the estimated advantage, and $\epsilon = 0.2$ is the clipping hyperparameter.

The value head is trained to minimise:

$$L^V(\phi) = E_t [(V_\phi(s_t) - G(d_t))^2]$$

The combined loss is:

$$L(\theta, \phi) = -L^{\text{CLIP}}(\theta) + c_1 \cdot L^V(\phi) - c_2 \cdot H[\pi_\theta]$$

where $H[\pi_\theta]$ is the entropy bonus (encouraging exploration) and $c_1 \approx 0.5$, $c_2 \approx 0.01$.

10. High-Level Solution Diagram

10.1 System Architecture

This is a system of three interdependent modules. Module 1 (Game Engine) maintains the game board state, the deck of cards and the rounds. Module 2 (RL Card Selection Agent) takes the game board state and three potential cards, and returns the selected card. Module 3 (Move Engine & Evaluation) gets the board with an updated list of legal moves for the selected card.

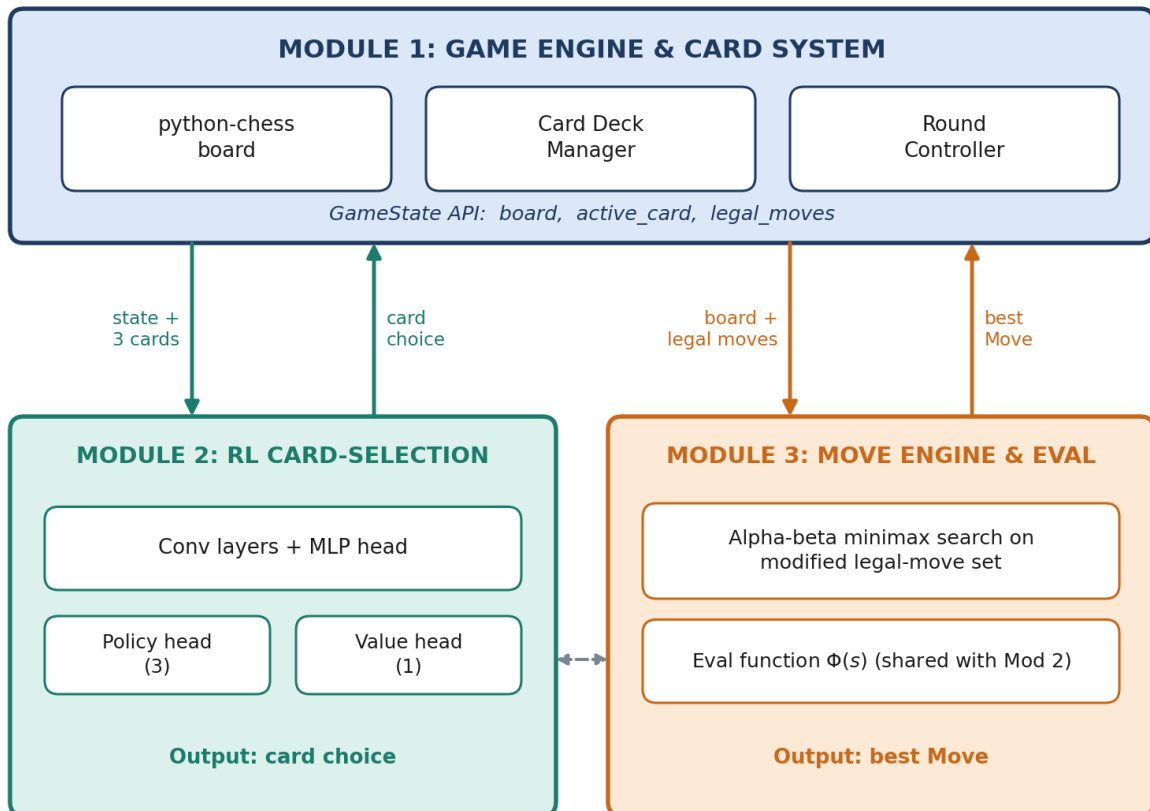


Figure 3. System architecture: three interconnected modules and the data exchanged between them.

10.2 Data Flow Per Round

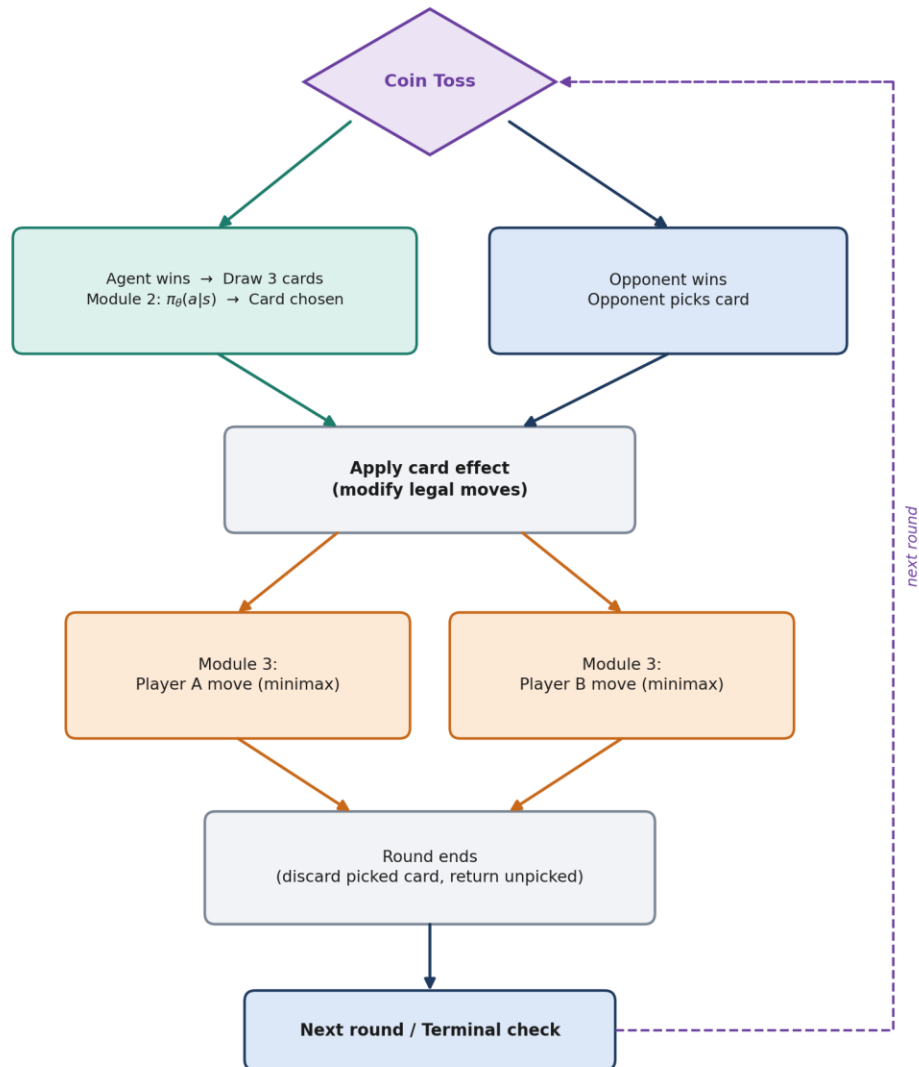


Figure 4. Data flow within a single round.

10.3 Neural Network Architecture

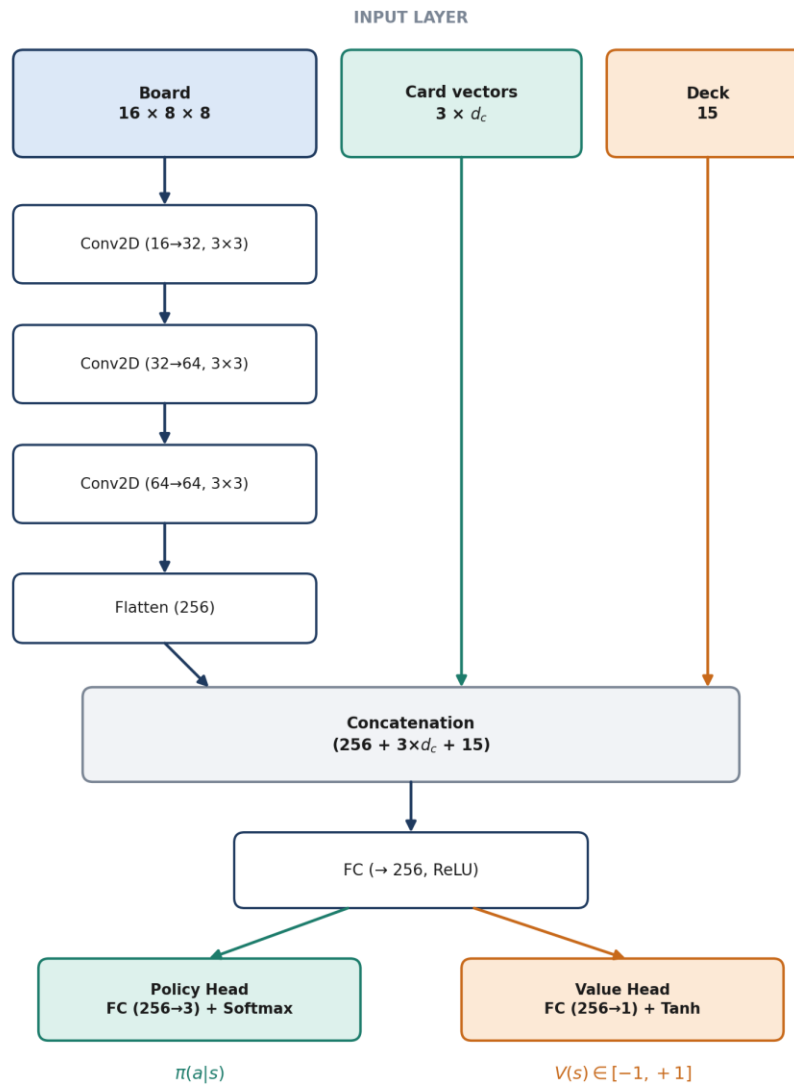


Figure 5. Shared-trunk policy/value network.

10.4 Training Loop (Self-Play)

1. Current Policy π_θ plays against itself; Module 3 performs the actions for both sides.
2. Gather samples $\{(s_t, a_t, G_t)\}$ from N full episodes ($N = 256$ per iteration).
3. Calculate Advantages: $\hat{A}_t = G(d_t) - V_\phi(s_t)$.
4. Updating θ and ϕ by optimizing the PPO objective function (clipped surrogates + value loss + entropy).
5. Policy is evaluated periodically against its previously saved copy and against a random card baseline.

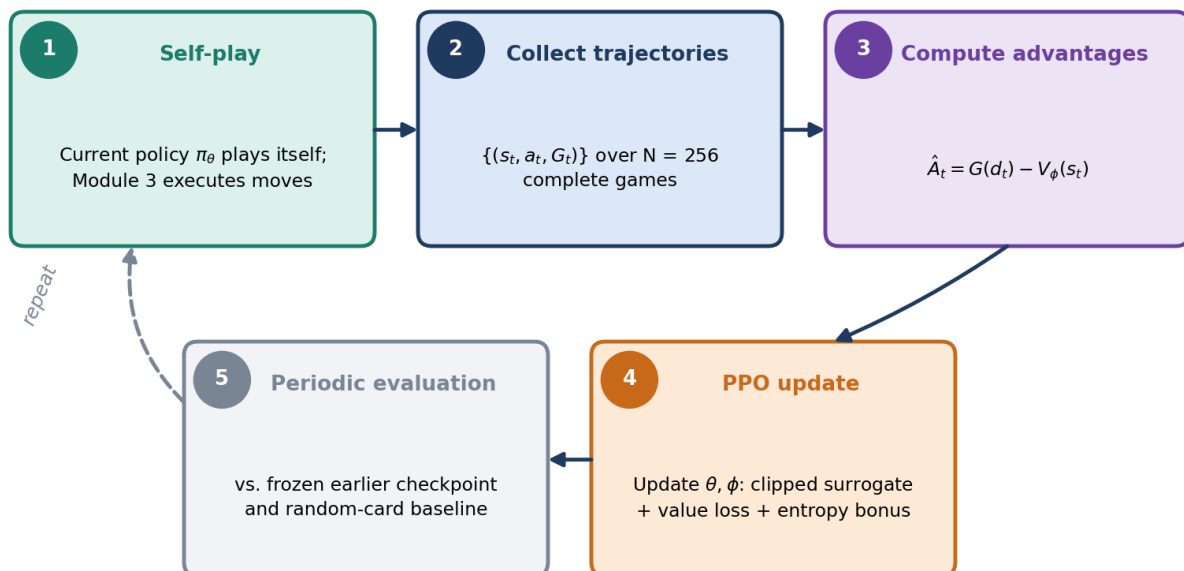


Figure 6. The self-play training cycle.

11. Dataset / Simulation Plan

11.1 No External Dataset — Pure Self-Play Generation

Card-Chess RL is a totally new version of a game; there is no dataset available from human plays. The training dataset is created via simulation of self-play, following the AlphaZero methodology. This corresponds to the reasoning given in Section 3 (Why Reinforcement Learning?); lack of labeled data is one of the driving factors behind RL.

11.2 Self-Play Data Generation Pipeline

Parameter	Value	Rationale
Games per training iteration	256	Balances sample diversity with compute time
Total training iterations	~200–500	Scaled to convergence of win-rate curve
Total games (projected)	~50,000–128,000	Sufficient for a 3-action policy to converge
Max rounds per game	200 (100 full move-pairs)	Prevents infinite games; draws declared if reached
Discount factor γ	0.97–0.99	Standard for episodic games with ~30–60 decisions
Shaping weight β	0.1–0.3	Tuned to prevent shaping from dominating terminal signal
PPO clip ϵ	0.2	Standard PPO hyperparameter
Learning rate	1×10^{-4} to 3×10^{-4}	Adam optimiser, with linear decay
Mini-batch size	64	Per PPO update epoch
PPO epochs per iteration	4	Standard PPO practice

11.3 Simulation Environment

The simulation environment is the Module 1 Game Engine itself, wrapped as a standard RL environment with the following interface:

```
class CardChessEnv:
    def reset(self) -> State:
        """Reset board to starting position, reshuffle deck."""

    def step(self, card_choice: int) -> tuple[State, float, bool, dict]:
        """
        Agent picks a card (0, 1, or 2).
        Internally: apply card, Module 3 plays both moves,
        advance rounds until agent's next pick or terminal.
        Returns: (next_state, shaped_reward, done, info)
        """

    def get_legal_cards(self) -> list[Card]:
        """Return the 3 currently offered cards."""
```

11.4 Data Collected Per Game

Each complete self-play game produces a trajectory of decision tuples:

$$\tau = \{(s_1, a_1, r_1, s_2), (s_2, a_2, r_2, s_3), \dots, (s_k, a_k, R_{\text{terminal}}, s_{\text{term}})\}$$

Across all games in one iteration, these tuples are aggregated into a replay buffer for PPO updates. Since PPO is an on-policy algorithm, the buffer is cleared after each training iteration.

11.5 Curriculum / Phased Training Plan

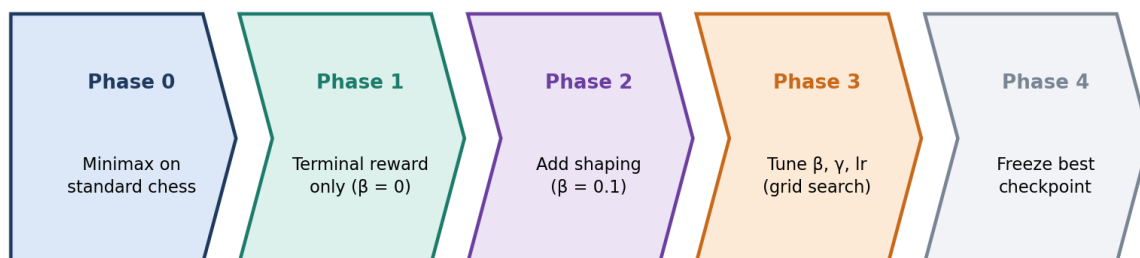


Figure 7. Phased training curriculum.

Phase	Description	Purpose
Phase 0	Train Module 3's minimax engine on standard chess (no cards)	Validate search correctness
Phase 1	Self-play with terminal reward only ($\beta = 0$)	Confirm the training pipeline works end-to-end

Phase	Description	Purpose
Phase 2	Add potential-based shaping ($\beta = 0.1$)	Accelerate learning with dense feedback
Phase 3	Tune β , γ , learning rate via grid search	Optimise final performance
Phase 4	Freeze best checkpoint for evaluation experiments	Produce final results

12. Evaluation Metrics and Baseline

12.1 Primary Metric

Win Rate (%) of the Trained RL Card-Selection Policy against Baseline Card Selection Policies, keeping the move engine (Module 3) fixed across all cases. This ensures that the effect of the trained card selection policy is isolated from that of the quality of the move generation.

Each comparison is based on 200 games, where the player plays 100 games as White and 100 games as Black, to take into account the advantage of playing first.

12.2 Baseline Strategies

Baseline	Description	Purpose
Random Picker	Selects one of the 3 offered cards uniformly at random	Lower-bound baseline; any competent policy should beat this
Always-First Picker	Always picks the first card presented	Tests whether the agent is learning position-dependent preferences vs. a fixed bias
Greedy Heuristic Picker	For each of the 3 candidate cards, simulates one ply with Module 3's evaluation function $\Phi(s)$ and picks the card yielding the highest $\Phi(s')$	Upper-bound heuristic baseline; shows whether RL's multi-step reasoning outperforms myopic optimisation

12.3 Secondary Metrics

Metric	Definition	Purpose
Training Win Rate Curve	Win rate of current policy vs. frozen checkpoint from iteration N-50, plotted over training iterations	Shows learning progress and convergence
Average Game Length	Mean number of rounds per game	Detects degenerate strategies (e.g. agent stalling)
Card Selection Entropy	$H[\pi_\theta] = -\sum_a \pi(a s) \log \pi(a s)$, averaged over evaluation games	Measures whether the agent develops strong card preferences vs. near-uniform selection
Shaping Reward Correlation	Pearson correlation between cumulative shaping reward and game outcome	Validates that Φ is a useful proxy for win probability

12.4 Statistical Testing

Significance tests for differences in win rates between the trained agent and each baseline include:

- Binomial proportion test (on win/loss results).
- Confidence intervals via bootstrapping (1,000 resamples) of the difference in win rate.

P value of $p < 0.05$ indicates significance.

12.5 Ablation Studies

Ablation	Modification	Tests
No shaping	Set $\beta = 0$; train with terminal reward only	Whether potential-based shaping accelerates learning
No deck encoding	Remove d_t from the state	Whether knowledge of remaining deck improves strategy
No value baseline	Use raw $G(d_t)$ instead of advantage $\tilde{A}_t$ in policy gradient	Whether the value head reduces variance meaningfully

13. Expected Novelty

13.1 Core Novel Contribution

Core contribution

The integration of per-round rule modification into a self-play RL training loop for a board game.

There are no previous works that incorporate all of the following into one single framework:

1. The use of an RL agent that chooses rule perturbations stochastically (card drafting).
2. The use of a classical search engine that is provided with a new list of legal moves every time because of the chosen rule.
3. Self-play based training that trains the agent not only how to take advantage of rule changes, but also how to predict rule changes from the opponent.

All three of these concepts exist independently of each other in the literature (self-play in AlphaZero for board games, CCG drafting, multi-variant search in Fairy-Stockfish).

13.2 Technical Novelty: Variable-Interval Potential-Based Reward Shaping

The classical potential-based shaping formula of Ng, Harada & Russell (1999) assumes the agent acts at every timestep:

$$r_{\text{shape}} = \gamma \cdot \Phi(s') - \Phi(s)$$

In Card-Chess RL, the agent only acts on rounds where it wins the coin toss. The number of game rounds between consecutive agent decisions is a random variable $\Delta t \sim \text{Geometric}(0.5)$. We adapt the shaping formula to:

$$r_{\text{shape}} = \gamma^{\Delta t} \cdot \Phi(s_{t+1}) - \Phi(s_t)$$

This ensures that the invariance condition of the original theorem is retained while properly discounting through the varying gap. As far as we know, this particular variant of the theorem has never been considered in the RL literature before.

13.3 Game Design Novelty

Card-Chess RL is an entirely new variant of the game that is currently unavailable on any chess platform (Lichess, Chess.com, Fairy-Stockfish). The idea of having a communal pool of cards that alter the rules of the game, in addition to a coin toss card picker, is unique and has never been considered before.

13.4 Empirical Novelty

The project provides the first empirically based analysis of three different methods of selecting cards in a game that changes its rules: learned strategy, heuristic strategy, and random selection of cards, similar to — but structurally distinct from — analyses of draft strategies found in CCG research (Vieira et al., 2019). What is important to mention is that the difference between CCGs and Card-Chess RL lies in the use of drafted cards in the next round with fixed rules.

14. References

- [1] Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., & Hassabis, D. (2018). A General Reinforcement Learning Algorithm that Masters Chess, Shogi and Go through Self-Play. *Science*, 362(6419), 1140–1144.
- [2] Ng, A. Y., Harada, D., & Russell, S. (1999). Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping. *Proceedings of the 16th International Conference on Machine Learning (ICML)*, 278–287.
- [3] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*.
- [4] Tesauro, G. (1995). Temporal Difference Learning and TD-Gammon. *Communications of the ACM*, 38(3), 58–68.
- [5] Browne, C. B., Powley, E., Whitehouse, D., Lucas, S. M., Cowling, P. I., Rohlfshagen, P., Tavener, S., Perez, D., Samothrakis, S., & Colton, S. (2012). A Survey of Monte Carlo Tree Search Methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1), 1–43.
- [6] Moravčík, M., Schmid, M., Burch, N., Lisý, V., Morrill, D., Bard, N., Davis, T., Waugh, K., Johanson, M., & Bowling, M. (2017). DeepStack: Expert-Level Artificial Intelligence in Heads-Up No-Limit Poker. *Science*, 356(6337), 508–513.
- [7] Vieira, R., Tavares, A. R., & Chaimowicz, L. (2019). Drafting in Collectible Card Games via Reinforcement Learning. *Proceedings of the 18th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 1865–1867.
- [8] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
- [9] Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D., & Kavukcuoglu, K. (2016). Asynchronous Methods for Deep Reinforcement Learning. *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, 1928–1937.
- [10] Nair, A., Srinivasan, P., Blackwell, S., Alcicek, C., Fearon, R., De Maria, A., Panneershelvam, V., Suleyman, M., Beattie, C., Petersen, S., Legg, S., Mnih, V., Kavukcuoglu, K., & Silver, D. (2015). Massively Parallel Methods for Deep Reinforcement Learning. *arXiv preprint arXiv:1507.04296*.
- [11] Fairy-Stockfish Contributors. (2021). Fairy-Stockfish: A Multi-Variant Chess Engine. <https://fairy-stockfish.github.io/>
- [12] Leela Chess Zero Contributors. (2018). Leela Chess Zero. <https://lczero.org/>